AIDE — System Report

Adaptive Intelligent Dynamic Embedding Framework

What the project does • what runs in the backend after an image is uploaded • what we are discovering

P. Yogishwar • 23261A6746 • B.Tech CSE (Data Science), MGIT Hyderabad • CD753PC Project Stage I

Repository: github.com/yogishwar09/aide-steganography

Metric	Measured value
Decision-engine training set	334 real BOSSbase images, ~342,000 image blocks
Method-classifier validation accuracy	0.877 (bpp regressor MAE ~0.14)
Version	AIDE 0.2 (robustness profiles, blind mode, GMD decoding, SPA/RS + ML steganalysis)
Embed time (one CPU core, 512×512)	~150 ms | Recover time ~40 ms | 0.16 ms/block inference
Decision-engine model size	13 KB
Stego quality by profile (30 covers)	quality 43.4 dB / 0.984 | balanced 40.1 / 0.967 | robust 35.0 / 0.912 | fortress 30.5 / 0.803 (PSNR / SSIM)
Report O4 (JPEG Q50 + Gaussian σ=20)	Met by the robust profile: 24/24 covers, BER 0
Automated tests	146 passing (no expected failures)
Hardware requirement	CPU only — no GPU / CUDA anywhere

All numbers in this document are measured from the running system and locked by an automated test — nothing here is an aspiration or an estimate.

1. What the project does

AIDE is a CPU-only image steganography system for defence and intelligence settings. It covers **both sides** of covert information transfer in a single tool:

1.1 Operator side — secure covert communication

Hide an encrypted message inside an ordinary-looking photograph so that interception reveals nothing, and so that the message can still be recovered after the image has been compressed, filtered, noised, cropped, or partially replaced in transit. This is the sender/receiver role: satellite & drone imagery relay, military secure communication, government document exchange.

1.2 Analyst side — blind threat detection (steganalysis)

Take an **arbitrary intercepted image** — no shared secret, no side-channel data, no assumption about whether or how it was modified — and decide whether it is likely carrying hidden data. This is the counter-intelligence role: border and port media screening, cyber-defence command-centre monitoring for covert data exfiltration.

1.3 How you use it

- **Command line:** `aide embed / extract / capacity / attack / evaluate / detect / benchmark / train / train-detector / profiles`
- **Web console** (`python webapp/app.py` or gunicorn / Docker): Operator, Analyst (single image and batch screening) and Model Insights pages.
- **REST API:** versioned `/api/v1/*` endpoints with an OpenAPI spec, optional API-key auth, JSON errors and request IDs.
- **Python API:** `aide.pipeline.embed() / recover()`, `aide.steganalysis.analyze()`.

The novelty is the **AIDE decision engine**: a lightweight trained model that decides, *per 8×8 or 16×16 image block*, four things at once — *where* to hide data, *how many* bits, *which* algorithm (LSB / DCT / DWT), and *whether* the block is reliable enough to use at all. No two images get the same embedding map.

2. What runs in the backend after you upload an image

Below is the exact sequence the web console / CLI executes. Each step names the module that does the work, so this doubles as a code map.

2.1 EMBED path (Operator → "Embed message")

#	Step	Module	What actually happens
1	Load & plane split	pipeline.load_image	Read the uploaded file's raw bytes. If colour, convert RGB→YCbCr and take only the Y (luma) plane; Cb/Cr are kept untouched for later recombination.
2	Block partition	features.partition_blocks	Split the plane into non-overlapping 8×8 blocks (16×16 for images ≥512 px). Each block gets a stable index used everywhere downstream.
3	Per-block features	features.extract_image_features	Four normalised signals per block: texture (Local Binary Pattern + GLCM energy), Shannon entropy, pixel variance, edge strength (Sobel + Canny).
4	AIDE decision engine	model.AIDEModel.decide	Trained decision tree (or tiny MLP) scores each block: $S(b)=\alpha\cdot$texture + $\beta\cdot$entropy + $\gamma\cdot$variance + $\delta\cdot$edge. Blocks below threshold τ are skipped; the rest are ranked and assigned a method and a bit-rate. The chosen *robustness profile* supplies the coefficient separation, RS strength and bpp cap.
5	Encrypt	crypto.encrypt	AES-256-GCM. Key = scrypt(shared-secret, image-hash) — memory-hard, so captured images cannot be brute-forced cheaply against a weak passphrase. A GCM authentication tag is attached.
6	Error-correction code	ecc.plan / ecc.encode	Reed-Solomon with profile-specific parity (e.g. 64 parity bytes per 255 for *balanced*). Short messages are split into ≥4 shortened codewords at the same parity ratio so interleaving can still spread damage.
7	Interleave	ecc.interleave	Scatter each codeword's bytes across many non-adjacent blocks, so a localised loss (a crop, a splice) damages a few bytes of many codewords instead of destroying whole codewords.
8	Adaptive embedding	embedding.dct (lsb / dwt optional)	Write the interleaved bits as mid-frequency DCT coefficient-pair QIM, preferring the 8 lowest robust pairs per block (relaxed only if the message would not fit). LSB/DWT blocks are left untouched by default — measured to cause ~40% of errors on ~2% of bits.
9	Tamper references	tamper.build_references	For every block actually used, store a SHA-256 hash and an 8-bin histogram in a sidecar JSON, for reference-based tamper localisation at recovery time.
10	Output	pipeline / webapp	Return the stego image (colour recombined if needed) plus the sidecar JSON (profile, RS parameters and KDF recorded), and report PSNR, SSIM and blocks used.

2.2 RECOVER path (Operator → "Recover")

The reverse of steps 8→5, driven entirely by the sidecar (no ML model needed at the receiver): recompute each block's hash and compare to the reference → a spatial *tamper map*; extract bits *with a soft reliability per bit* (the coefficient-pair margin); reverse the interleave; Reed-Solomon decode with tamper-flagged blocks as known erasures and **GMD reliability-guided decoding** (the least reliable bytes are tried as erasures, and the candidate with the smallest reliability-weighted distance wins, up to 2t errors instead of t); AES-256-GCM decrypt and verify the tag. Tag failure = "tampered beyond recovery".

2.2b BLIND mode (no sidecar)

A self-describing alternative: blocks are visited in a keyed pseudo-random order derived from the secret; a 14-byte header (profile, random salt, length) is RS-protected and stored three times with soft combining; the AIDE score sets each block's embedding *strength*, which the receiver does not need because extraction reads only the sign of each pair difference. The receiver needs only the image and the secret; a wrong secret finds no header at all.

2.3 DETECT path (Analyst → "Analyze")

Completely independent of the sidecar. Takes only the image:

- **Chi-square pairs-of-values test** (Westfeld & Pfitzmann, 2001) — the primary signal. LSB embedding of an encrypted (near-random) payload equalises the counts of value pairs (2k, 2k+1); natural photos do not have this. Run on the whole image and per horizontal row-band.
- **LSB / next-bit-plane correlation** — a real photo's least-significant bit plane is weakly correlated with its finer structure; overwriting it with random payload collapses that correlation toward zero.
- **LSB bit-balance** — deviation of the LSB plane from a 50/50 split (weakest signal, low weight).
- **Sample Pairs Analysis + RS analysis** — quantitative estimators of the embedded fraction, validated against known embedding rates; they catch randomly scattered LSB payloads chi-square misses and report an estimated payload size.
- **Trained ML steganalyser** — SPAM + DCT coefficient-pair features, gradient boosting, with an IsolationForest guard that caps verdicts on images unlike its training covers.
- Combined into a verdict: *clean* / *suspicious* / *likely steganographic* (most severe of the statistical and ML verdicts).

3. What we are discovering (measured findings)

The project's method is "measure, don't assume": every claim below is backed by a test on real BOSSbase images, and several of the original report's assumptions did **not** survive contact with measurement.

3.1 Measured recovery by robustness profile

Bit-exact, AES-GCM-verified recovery on 30 BOSSbase covers, 156-byte message (`aide benchmark`, reports/benchmark/benchmark.md):

Attack	quality	balanced	robust	fortress
JPEG Q50 (O4) / Q30	100% / 20%	100% / 100%	100% / 100%	100% / 100%
Gaussian noise σ=10 / σ=20 (O4)	0% / 0%	100% / 0%	100% / 100%	100% / 100%
Salt & pepper 1%	0%	0%	100%	100%
Median 3×3 / Gaussian blur	52% / 100%	100% / 100%	100% / 100%	100% / 100%
Rescale 0.75× / 0.5×	100% / 40%	100% / 92%	100% / 96%	100% / 100%
Splice 5% / Crop 10%	100% / 84%	100% / 88%	100% / 100%	100% / 100%
Mean PSNR / SSIM	43.4 dB / 0.984	40.1 / 0.967	35.0 / 0.912	30.5 / 0.803

For comparison, AIDE 0.1's fixed parameters on the same covers: σ=20, salt & pepper and 0.5× rescale 0%, median 69%, splice 5% 85%, crop 10% 50% (PSNR 39.7 dB). Wrong secrets and tampered ciphertext are always rejected; colour covers keep full chroma fidelity.

3.2 Finding: the DWT method as originally conceived did not work

Single-level Haar DWT embedding in the horizontal/vertical *detail* sub-bands measured **~0% bit survival** against median filtering and geometric scaling — at *any* coefficient separation tested, up to delta = 400 (well past visible distortion). Both attacks resample the exact 2×2 neighbourhood a single-level Haar transform depends on. This directly contradicts the report's assumption that DWT gives "superior robustness against geometric attacks". **Fix:** embed in the *coarse* 2-level approximation sub-band instead (a twice-averaged, low-frequency view of the block). Measured result: 100% survival against JPEG Q30/50, median filtering, and 2× downscale-then-restore, at delta = 40 and a *better* per-block PSNR (~37 dB vs ~29 dB) than the broken

version needed.

3.3 Finding: DCT has a hard capacity–versus–robustness trade-off

The more DCT capacity a block is asked for, the further into higher, less-robust mid-frequencies it must reach. Measured on 16×16 blocks against a 3×3 median filter: 13 coefficient pairs → 20/20 blocks survive; 26 pairs → 10/20; 39+ pairs → 0/20 — and raising the coefficient separation does **not** help, because it is about *which* frequencies are used, not how far apart the pair is pushed. **Fix:** cap DCT to a measured-safe frequency band. Cost: less capacity per block, so more blocks are needed — which surfaces as an honest "capacity insufficient" error rather than a silent recovery failure.

3.4 Finding: LSB is fragile to essentially everything

Measured ~45–55% bit-error rate under every attack tried, including a plain re-save. LSB depends on exact pixel values. Its error rate is so much higher than DCT/DWT's that even 1% of the payload coming from LSB was measured to flip an otherwise-passing recovery into a failure. **Fix (0.2):** per-method attribution showed LSB and DWT causing ~40% of median-filter bit errors on ~2% of the bits, so every profile now routes the payload to DCT only; zeroing the budgets raised 10%-crop recovery from 52% to 88% at *better* PSNR. The engine still decides where, how much and which method; LSB/DWT blocks are left untouched.

3.5 What 0.2 changed, and what still does not work

- **Solved in 0.2:** σ=20 noise and salt-and-pepper (robust profile), 0.5× rescaling (fortress 100%, balanced 92%), median filtering (100% from balanced up), single-codeword messages (shortened RS codewords), and the mandatory sidecar (blind mode).
- **Targeted detectability** — see 3.6: a trained detector catches AIDE. Fixing it needs side-informed embedding (e.g. syndrome-trellis codes); not attempted.
- **Geometric desynchronisation** — rotation and true (canvas-changing) cropping misalign the block grid; no registration template yet.
- **Capacity vs robustness** — messages near capacity lose the low-frequency layout preference and most 0.5×-rescale survival.
- **Scale** — trained on 334 BOSSbase images (not ALASKA2); CNN baselines (O7) and Raspberry Pi timings not measured.

3.6 Finding: the detection side, and what it says about AIDE's own stealth

Test	Measured outcome
SPA / RS accuracy vs ground truth	True rate 0.25 → 0.27 / 0.28; 0.50 → 0.51 / 0.53; clean covers ≈ 0.03
Clean images falsely flagged (held out)	1 in 40 (2.5%), thresholds calibrated on 200 disjoint covers
Randomly scattered LSB at a 10% rate	40 / 40 detected (chi-square alone: ~0)
AIDE stego vs classical detectors	0 of 15 flagged by chi-square, SPA or RS
AIDE stego vs the trained ML detector	100% detected for every profile and mode, AUC 0.998–1.000, 1% false positives (100 held-out covers)

The honest conclusion: AIDE meets the report's "negative detectability" claim against classical blind tools only. Coefficient-pair QIM forces $|c_i - c_j| \geq \delta$, leaving a deficit of small pair differences that a targeted detector learns immediately. A second finding came from the same detector: trained on photographs, it scored a clean synthetic pattern at P(stego)=0.99 (cover-source mismatch), which is why an out-of-distribution guard now caps such verdicts.

3.7 A finding about method itself

A false-positive regression in the detector was caught *by testing* before shipping: an early scoring change that folded a per-row-band breakdown into the verdict amplified 1/40 clean images being flagged into 16/20. It was reverted, and the row-band data kept as a labelled “diagnostic only” display. The general pattern — a plausible improvement that measurement rejects — recurred often enough to be the project's main working lesson. In 0.2 it recurred again: the first GMD decoder accepted the first successful decode and was measured to *mis-correct* on short codewords (all 13 failures of a 20-trial test were wrong-but-valid codewords, caught by the GCM tag); minimum-distance selection plus saturation-aware reliability took that test to 20/20.

4. System facts

4.1 The trained decision engine

Training is empirical, not circular: for each block, method labels come from the report's own texture/edge rule (with thresholds set from data percentiles), and the bit-rate label comes from actually embedding random bits at each level and *measuring* the resulting block PSNR. A DecisionTreeClassifier learns method; a DecisionTreeRegressor learns bit-rate; the classifier's normalised feature importances become the explainable weights ($\alpha,\beta,\gamma,\delta$). Measured learned weighting: texture ≈ 0.55, edge ≈ 0.44, entropy and variance ≈ 0.01 combined — i.e. the model relies almost entirely on texture and edge content.

4.2 Cryptography & coding parameters

Component	Parameter
Cipher	AES-256 in GCM mode (FIPS 197), 12-byte nonce, 16-byte tag
Key derivation	scrypt(shared secret, SHA-256(cover bytes)), $N=2^{14}$, r=8, p=1; SHA-256 (report formula) for 0.1 files
Error-correcting code	Reed-Solomon, parity per 255 bytes: 64 / 64 / 104 / 176 by profile; shortened codewords for short messages; GMD reliability-guided decoding
Interleaving	Column-major over the codeword matrix — adjacent embedded bytes belong to different codewords
DCT embedding	Mid-frequency zigzag coefficient pairs, relative-magnitude QIM, separation δ = 20 / 36 / 64 / 80 by profile, preferring the 8 lowest robust pairs per block
DWT embedding	2-level Haar, coefficient pairs in the coarse approximation sub-band (available; payload budget 0 by default)
Blind-mode header	14 bytes (magic, version+profile, 8-byte salt, length), RS(40,18), 3× soft-combined

4.3 Attack simulation library

Eight concrete simulations across the report's six categories: JPEG compression, Gaussian noise, salt-and-pepper noise, cropping (fixed-canvas region removal), splicing, median filter, Gaussian filter, and geometric scaling (downscale-then-restore). All operate on the luma plane and preserve image dimensions so block alignment for recovery stays valid.

4.4 Technology stack

Python 3, CPU only. NumPy, OpenCV, scikit-learn, scikit-image, SciPy, PyWavelets, PyCryptodome, reedsolo, Pillow for the core; Flask + gunicorn for the console and REST API; pytest for the 146-test suite; ruff, Docker and GitHub Actions CI for engineering. No GPU, CUDA, or proprietary hardware anywhere — deployable on a Raspberry Pi 4 / Jetson-class board, as the report intends.

Generated from the running AIDE 0.2 system on 2026-10-11. Full source, tests, and the honest “what works / what doesn't” breakdown: github.com/yogishwar09/aide-steganography (see README.md).